Ministry of science and higher education of the Russian Federation
Southern Federal University

COURSE SYLLABUS FOR

Algorithmization and programming

Подписано электронной подписью:

Г. Е. Веселов, директор Института компьютерных технологий и информационной безопасности

Сертификат №0643656F00F3B21B9D460D25A94BE5A817
действителен с 5 июня 2025 г. по 5 июня 2026 г.

Author (s) of the work program:

N.Sh. Khusainov, PhD, Associate Professor, Head of the Department of Mathematical Support and Computer Applications of the Institute of Computer Technologies and Information Security

I.G. Danilov, Ph.D. Associate Professor, software developer, Auriga LLC

A.M. Mansour, Ph.D. Senior lecturer, programmer, of the Institute of Computer Technologies and Information Security

The Syllabus has been approved at the meeting of the Educational and Methodological Council of the Institute of Computer Technologies and Information Security

June 19, 2025, Minutes № 4

CONTENT

I. COURSE AIMS AND OBJECTIVES

Course (module) aims:

– To provide an understanding and knowledge of computer information representation methods, modern algorithmic languages and their application areas, and the stages of software application development; basic algorithms and data structures;

– To teach the use of modern methods and tools for algorithm and program development, structured programming techniques, algorithm notation methods in a high-level language, and program debugging methods.

Objectives:

– To study program development tools and their components, as well as algorithmization methodology;

– To acquire practical skills in using the C/C++ programming language, program debugging, and program documentation.

II. COURSE SEQUENCING

This course is part of the compulsory professional disciplines' module of the educational program.

This course builds on the basic knowledge, skills, and abilities developed during the previous level of education.

The knowledge, skills, and abilities developed in this course will be required for mastering the following elements of the educational program:

– Object-oriented programming;

– Operating systems;

– Database management systems.

III. INTENDED LEARNING OUTCOMES

The aim of the course is to achieve the following learning outcomes:

Code of the field of study, specialty	**Competence**	**Indicators**	**Learning outcomes**
09.03.01 09.03.02 09.03.03 09.03.04	CPC-3 Capable of participating in the development of regulatory, technical and reporting documentation, presenting the results of professional activities using standards, norms and rules	CPC-3.1 Prepares regulatory, technical and reporting documentation, taking into account standards, norms and rules	**To know**: - rules for the design of elements of software documentation and flowcharts of algorithms in accordance with the requirements of the Unified System of Software Documentation. **To be able to**: - prepare elements of software documentation in accordance with the requirements of state and local regulatory documents. **To apply**: - skills in preparing reports on created algorithms and programs
09.03.01 09.03.02 09.03.03	CPC-4. Capable of developing algorithms and programs suitable for practical application in professional activities.	CPC-4.1 Selects data structures and develops algorithms for solving problems of professional activity CPC-4.2 Develops a software implementation of the algorithm CPC-4.3. Performs debugging and testing of programs	**To know**: - typical algorithms and data structures; - principles of creating a software implementation of an algorithm, syntax and semantics of the C/C++ programming languages; - dependence of the complexity of software implementation of algorithms on the complexity of the data structure; - methods of presenting and coding various types of information; - methods of debugging and testing programs;

Code of the field of study, specialty	**Competence**	**Indicators**	**Learning outcomes**
			To be able to: - apply standard algorithms and data structures to solve a given problem; - represent numerical data in various codes, perform operations on them; - create program code, implement individual fragments of complex programs to solve professional problems; - use the principles of a structured approach to create programs; - debug programs in C/C++; **To possess**: - skills for developing algorithms for solving problems of professional activity , assessing the complexity of the algorithm - skills in using a modern programming environment and using software libraries for program development; - skills in using a modern development environment for debugging program code;
09.03.04	CPC-4. Capable of developing algorithms and programs suitable for practical use, and to design, build, and test software products and software complexes across various fields of human activity	CPC-4.1 Selects data structures and develops algorithms for solving problems of professional activity CPC-4.2 Develops a software implementation of the algorithm CPC-4.3. Performs debugging and testing of programs	**To know**: - typical algorithms and data structures; - principles of creating a software implementation of an algorithm, syntax and semantics of the C/C++ programming languages; - dependence of the complexity of software implementation of algorithms on the complexity of the data structure;

Code of the field of study, specialty	**Competence**	**Indicators**	**Learning outcomes**
			- methods of presenting and coding various types of information; - methods of debugging and testing programs; **To be able to**: - apply standard algorithms and data structures to solve a given problem; - represent numerical data in various codes, perform operations on them; - create program code, implement individual fragments of complex programs to solve professional problems; - use the principles of a structured approach to create programs; - debug programs in C/C++; **To possess**: - skills for developing algorithms for solving problems of professional activity , assessing the complexity of the algorithm - skills in using a modern programming environment and using software libraries for program development; - skills in using a modern development environment for debugging program code;
09.03.04	CPC-6 is capable of searching, storing, processing and analyzing information from various sources and databases, presenting it in the required format using information, computer and network technologies.	CPC-6.2 Selects the method of presentation, organizes the storage and processing of information using computer technologies	**To know**: - distinctive features of fundamental and user data types and program objects used for storing and processing various types of information. **To be able to**:

Code of the field of study, specialty	**Competence**	**Indicators**	**Learning outcomes**
			- select fundamental data types and create user-defined data types that correspond to the information processing algorithms in the programs being developed **To possess**: - develop and debug programs taking into account the specifics of operations for processing software objects of various types and access.
09.03.01 09.03.02 09.03.03 09.03.04	PC-10. PL-3 Able to use C/C++ programming languages to solve AI problems	PC-10.1. PL-3.1 Develops and debugs efficient multithreaded solutions in C++, tests, tests, and evaluates the quality of such solutions.	**To know**: Basic concepts related to the parallel execution of programs and processes, the differences between a thread and a process, principles of organizing data access and interaction between multithreaded applications, possible problems and solutions **To be able to**: use thread synchronization mechanisms based on semaphores and mutexes **To possess**: software tools (library functions) for working with threads (creation, destruction, synchronization of threads) and techniques for debugging multithreaded applications
		PC-10.2. PL-3.2 Develops and debugs AI systems in C++ for specific hardware platforms with limitations in computing power, including embedded systems	**To know**: Basic classification algorithms (linear regression, Bayesian classifier, K-means, decision tree) and features of their software implementation on x86 and x86-64 platforms **To be able to**: implement and debug programs using basic classification algorithms and evaluate their performance on various computing platforms **To possess**:

Code of the field of study, specialty	**Competence**	**Indicators**	**Learning outcomes**
			tools and techniques for developing and debugging program code for x86, x86-64 platforms
		PC-10.3. PL-3.3 Develops and debugs solutions in C++ that use GPUs and FPGAs for massive parallelization of computations within a general AI system, using both ready-made solutions and developing their own	**To know**: principles of organizing parallel computing on GPUs and the capabilities of software libraries for organizing parallel computing **To be able to**: Use software libraries and APIs to massively parallelize GPU computing in your projects **To possess**: tools for developing programs with support for massive parallelization for GPUs

IV. COURSE CONTENT AND STRUCTURE

Course/student load is 6 credits, 216 hours,
Including 1 credit, 36 hours per exam

Form of intermediate attestation: exam

4.1. The content of the course, structured by topics (educational meetings)

4.1.1 The content of the course, structured by topics (educational meetings) for 1st semester of full-time education

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact hours				Independent work				
		Lectures	Practical	Laboratory	Consultations					
Module 1. Fundamentals of information representation and processing in a personal computer. Algorithmization. Basic data structures and data processing algorithms. Algorithms										
1.	Lecture 1_1. **Introductory Lecture.** Course Subject and Structure. Number Systems	2					Lecture-visualization			
2.	Lecture 1-2: **Information representation in computers.** Coding of numerical and symbolic information in computers and basic operations.	2					Lecture-visualization			
3.	Lecture 1_3 **Algorithms.** Concept, Properties, and Description Methods.	2					Lecture-visualization			
4.	Practice 1_1 **Integer Information Representation in Computers.** Problem Solving		2				Seminar			

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact hours				Independent work				
		Lectures	Practical	Laboratory	Consultations					
Module 2. C/C++ Basics. Input/Output. Linear Algorithms										
5.	Lecture 2_1 **General information about the C language.** Developing and debugging programs in C/C++	2					Lecture-visualization			
6.	Practice 1_2 **Representation of String Information and FP Numbers in Computers.** Problem Solving		2			4	Seminar	Homework 1_1		
7.	Lecture 2_2 **Data Types.** Description of Types. Operators. Operations	2					Lecture-visualization			
8.	Practice **1_3 Flowcharts. Linear Algorithms and Branching.** Problem Solving		2				Seminar	Test 1		
9.	Lab 0 **Integrated tools** for developing and debugging C / C++ programs			2			Reproductive laboratory work	laboratory work No. 0		
10.	Lecture 2_3 **Computing Operations in C**	2					Lecture-visualization			
11.	Lecture 2_4 **Input/Output Organization**	2					Lecture-visualization			
12.	Practice **1_4 Flowcharts. Cycles and Subroutines.** Problem Solving		2			4	Seminar	Homework 1_2		
Module 3. Branching and Loops										
13.	Lecture 3 **Branching and Cycles**	2					Lecture-visualization			

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact hours				Independent work				
		Lectures	Practical	Laboratory	Consultations					
Module 4. Arrays. Pointers. Strings										
14.	Lecture 4_1 **Arrays. Pointers. Strings.** Arrays.	2					Lecture-visualization			
15.	Practice 2_1 **Linear Algorithms**. Data Types. Operators. Operations. Expressions		2			3	Seminar	Homework 2_1		
16.	Laboratory work 1. **Presentation of information in a computer and the basics of algorithmization.** Final lesson on topic 1.			2		6	Reproductive laboratory work	laboratory work No. 1	*Completing the laboratory work*	11
17.	Lecture 4_2 **Arrays. Pointers. Strings.** Pointers. Strings	2					Lecture-visualization			
Module 5. Subroutines										
18.	Lecture 5 **Subroutines**	2					Lecture-visualization			
19.	Practice 2_2 **Linear Algorithms.** Data Types. Formatted Input/Output		2			3	Seminar	Homework 2_2 Test 2		
Module 6. Working with Files										
20.	Lecture 6: **Working with Files**. Text and Binary Files	2					Lecture-visualization			
Module 7. User-Defined Data Types										
21.	Lecture 7: **User-defined data types. Dynamic memory management.**	2					Lecture-visualization			
22.	Practice 3_1 **Branching and Looping.** Conditional Operator and Switch		2			3	Seminar	Homework 3_1		

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact hours				Independent work				
		Lectures	Practical	Laboratory	Consultations					
23.	Lab 2: **Linear Algorithms.** Final Lesson on Topic 2			2		4	Reproductive laboratory work	laboratory work No. 2	*Completing the laboratory work*	7
Module 8. MISCELLANEOUS										
24.	Lecture 8_1 Preprocessor. **Development of multi-module programs and libraries. Build systems. Standard Libraries** (Mathematical Functions, Symbolic Functions, String Functions, Time Functions) Internationalization	2					Lecture-visualization			
25.	Lecture 8_2 **Standard data structures (stack, queue, linked lists, trees) Standard data processing algorithms (sorting, searching, hashing).** Evaluation of algorithm complexity.	2					Lecture-visualization			
26.	Practice **3_2 Branching and Looping.** Loops with a Counter and a Condition		2			3	Seminar	Homework 3_2 Test 3		
27.	**Lecture 8_3**: **Multithreaded programming** in C++, testing multithreaded programs. Libraries.	2					Lecture-visualization			

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact hours				Independent work				
		Lectures	Practical	Laboratory	Consultations					
28.	**Lecture 8_4 Development and debugging of AI algorithms** for platforms with limited computing resources. Linear regression, Bayesian classifier, K-means, decision tree and features of their software implementation on various hardware platforms	2					Lecture-visualization			
29.	Practice 4_1 **Arrays. Pointers. Strings.** Arrays and Pointers		2			3	Seminar	Homework 4_1		
30.	Lab 3: **Branching and Looping.** Final Lesson on Topic 3			2		4	Reproductive laboratory work	laboratory work No. 3	*Completing laboratory work*	7
31.	Lecture 8_5 **Programming, capabilities and limitations** of program execution on GPUs and FPGAs	2					Lecture-visualization			
32.	Practice 4_2 **Arrays. Pointers. Strings.** Strings and Pointers		2			3	Seminar	Homework 4_2 Test 4		
33.	Practice 5_1 **Subroutines.** Data transfer and return.		2			3	Seminar	Homework 5_1		
34.	Lab 4: **Arrays, Pointers, and Strings.** Final Lesson on Topic 4			2		4	Reproductive laboratory work	laboratory work No. 4	*Completing the laboratory work*	7
35.	Practice 5_2 **Subroutines.** Recursive subroutines and library functions		2			3	Seminar	Homework 5_2 Test 5		
36.	Practice 6_1 **Files.** Text files		2			3	Seminar	Homework 6_1		
37.	Laboratory work 5 **Subroutines** Final lesson on topic 5			2		4	Reproductive laboratory work	laboratory work No. 5	*Completing the laboratory work*	7
38.	Practice 6_2 **Files.** Binary files		2			3	Seminar	Homework 6_2 Test 6		

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact hours				Independent work				
		Lectures	Practical	Laboratory	Consultations					
39.	Practice 7_1 **Dynamic memory and user-defined data types.** Dynamic memory. The struct type.		2			3	Seminar	Homework 7_1		
40.	Lab 6 **Files.** Final lesson on topic 6			2		4	Reproductive laboratory work	laboratory work No. 6	*Completing the laboratory work*	7
41.	Practice 7_2 **Dynamic memory and user-defined data types. The** union type. The typedef operator.		2			3	Seminar	Homework 7_2 Test 7		
42.	Practice 8_1 **Implementation of a linked list.**		2			3	Seminar	Homework 8_1		
43.	Lab 7: **Dynamic Memory and User-Defined Data Types.** Final Lesson on Topic 7			2		4	Reproductive laboratory work	laboratory work No. 7	*Completing the laboratory work*	7
44.	Practice 8_2: **Linked List Implementation. Completion**		2			3	Seminar	Homework 8_2 Test 8		
45.	Lab 8. **Solving AI problems based on multithreaded programming and heterogeneous computing.**			2		10	Reproductive laboratory work	Case study implementation	*Case study implementation*	7
Summative Assessment		-	-	-	-	36		Exam questions and tickets for semester 1	*exam*	40
Total hours		**36**	**36**	**18**		**126**		**-**		**100**

4.1.2 Contents of the discipline, structured by topics (study meetings) for the 3 trimester of correspondence education

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact work				Independent work				
		Lectures	Practical	Laboratory	Consultations					
1.	**Introductory lecture.** Course subject and structure. Number systems. Information coding. Development and debugging of programs in C/C++ **Independent study of sections of the discipline.** Coding of numerical and symbolic information in computers and basic operations. Concept, properties and methods of description. Development and debugging of programs in C/C++ Description of types. Operators. Operations. Computing operations in C	2				34	Lecture-visualization			
Total hours		**2**				**34**		**-**		

4.1.3 Contents of the discipline, structured by topics (study meetings) for the 4 trimester of correspondence education

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact work				Independent work				
		Lectures	Practical	Laboratory	Consultations					
1.	**Independent study of sections of the discipline.** Input/output organization Branching and loops Arrays. Pointers. Strings. Subroutines					72				
Total hours		**0**				**72**		**-**		

4.1.4 Contents of the discipline, structured by topics (study meetings) for the 5 trimester of correspondence education

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact work				Independent work				
		Lectures	Practical	Laboratory	Consultations					
1.	Working with files. Text and binary files. User-defined data types. Dynamic memory management. Memory management. Scope and lifetime. Preprocessor. Development of multi-module programs and libraries. Build systems. Standard libraries.	2				20	Lecture-visualization			
2.	Practice 1. **Representation of integer information in a computer.** Problem solving		2			10	Seminar	Homework 1	*Submitting homework*	12

No p/n	Topic of the lesson and issues under consideration	Types of academic work and their labor intensity, hours (including the use of online courses)					Format of the lesson	Name of the assessment tool	Type of control	Sum of points per lesson
		Contact work				Independent work				
		Lectures	Practical	Laboratory	Consultations					
3.	Lab 1. **Basics of Algorithmization. Construction of Flowcharts.**			2		15	Reproductive laboratory work	laboratory work No. 1	*Completion and defense of laboratory work*	7
4.	Practice 2: Loops, **Arrays, Pointers.**		2			22	Seminar	Homework 2 Test	*Submitting homework* *Performing the test*	12 15
5.	Lab 2. **Branching and loops.**			2		15	Reproductive laboratory work	laboratory work No. 2	*Completion and defense of laboratory work*	7
6.	Lab 3. **Arrays. Pointers. Strings.**			2		15	Reproductive laboratory work	laboratory work No. 3	*Completion and defense of laboratory work*	7
Interim assessment		-	-	-	-	9		Exam questions and tickets for trimester 5	*exam*	40
Total hours		**2**	**4**	**6**		**76**		**-**		**100**

4.2. Plan for extracurricular independent work

No p/n	Topic of the lesson	Semester	Completion time (weeks)	Type of independent work	Time Expenditure (Hours)	Educational and methodological support
Module 1. Fundamentals of Information Representation and Processing in a Computer. Algorithmization. Basic Data Structures and Data Processing Algorithms. Algorithms						
1.	Practice 1_2 **Representation of string information and numeric values in a computer** . Problem solving	1	2	– doing homework	4	[1]-[3], [6]-[8]
2.	Practice **1_4 Flowcharts. Cycles and Subroutines.** Problem Solving	1	4	– doing homework	4	[1]-[3], [6]-[8]
3.	Laboratory work 1. **Presentation of information in a computer and the basics of algorithmization.** Final lesson on topic 1.	1	5	– review and revision of lecture material; – preparation for laboratory work	6	[1]-[3], [6]-[8]
Module 2. C/C++ Basics. Input/Output. Linear Algorithms						
4.	Practice 2_1 **Linear Algorithms** . Data Types. Operators. Operations. Expressions	1	5	doing homework	3	[1]-[3], [6]-[8]
5.	Practice 2_2 **Linear Algorithms.** Data Types. Formatted Input/Output	1	6	doing homework	3	[1]-[3], [6]-[8]
6.	Lab 2: **Linear Algorithms.** Final Lesson on Topic 2	1	7	– elaboration and repetition of lecture material; preparation for laboratory work	4	[1]-[3], [6]-[8]
Module 3. Branching and Loops						
7.	Practice 3_1 **Branching and Looping.** Conditional Operator and Switch	1	7	doing homework	3	[1]-[3], [6]-[8]
8.	Practice **3_2 Branching and Looping.** Loops with a Counter and a Condition	1	8	doing homework	3	[1]-[3], [6]-[8]
9.	Lab 3: **Branching and Looping.** Final Lesson on Topic 3	1	9	– review and revision of lecture material; preparation for laboratory work	4	[1]-[3], [6]-[8]
Module 4. Arrays. Pointers. Strings						

No p/n	Topic of the lesson	Semester	Completion time (weeks)	Type of independent work	Time Expenditure (Hours)	Educational and methodological support
10.	Practice 4_1 **Arrays. Pointers. Strings.** Arrays and Pointers	1	9	doing homework	3	[1]-[3], [8]-[10]
11.	Practice 4_2 **Arrays. Pointers. Strings.** Strings and Pointers	1	10	doing homework	3	[1]-[3], [8]-[10]
12.	Lab 4: **Arrays, Pointers, and Strings.** Final Lesson on Topic 4	1	11	– elaboration and repetition of lecture material; preparation for laboratory work	4	[1]-[3], [8]-[10]
Module 5. Subroutines						
13.	Practice 5_1 **Subroutines.** Data transfer and return.	1	11	doing homework	3	[1]-[3], [8]-[10]
14.	Practice 5_2 **Subroutines.** Recursive subroutines and library functions	1	12	doing homework	3	[1]-[3], [8]-[10]
15.	Laboratory work 5 **Subroutines** Final lesson on topic 5	1	13	– review and revision of lecture material; preparation for laboratory work	4	[1]-[3], [8]-[10]
Module 6. Working with Files						
16.	Practice 6_1 **Files.** Text files	1	13	doing homework	3	[1]-[3], [8]-[10]
17.	Practice 6_2 **Files.** Binary files	1	14	doing homework	3	[1]-[3], [8]-[10]
18.	Lab 6 **Files** . Final lesson on topic 6	1	15	– review and revision of lecture material; preparation for laboratory work	4	[1]-[3], [8]-[10]
Module 7. User-Defined Data Types						
19.	Practice 7_1 **Dynamic memory and user-defined data types.** Dynamic memory. The struct type.	1	15	doing homework	3	[1]-[3], [8]-[10]
20.	Pratika7_2 **Dynamic memory and user-defined data types.** Union type. Typedef operator.	1	16	doing homework	3	[1]-[3], [8]-[10]

No p/n	Topic of the lesson	Semester	Completion time (weeks)	Type of independent work	Time Expenditure (Hours)	Educational and methodological support
21.	Lab 7: **Dynamic Memory and User-Defined Data Types.** Final Lesson on Topic 7	1	14	– review and revision of lecture material; preparation for laboratory work	4	[1]-[3], [8]-[10]
Module 8. MISCELLANEOUS						
22.	Practice 8_1 **Implementation of a linked list.**	1	17	doing homework	3	[1]-[3], [8]-[10]
23.	Practice 8_2: **Linked List Implementation. Completion**	1	18	doing homework	3	[1]-[3], [8]-[10]
24.	Lab 8. **Solving AI problems based on multithreaded programming and heterogeneous computing.**	1	19	– review and revision of lecture material; preparation for laboratory work	10	[5]-[6], [11]-[14]
Preparing for the exam					**36**	**[1]–[10]**
Total workload of independent work on the subject					**126**	**-**

4.3. Plan for extracurricular independent work for correspondence courses

No p/n	Topic of the lesson	Trimester	Completion time (weeks)	Type of independent work	Time Expenditure (Hours)	Educational and methodological support
1.	**Introductory lecture.** Course subject and structure. Number systems. Information coding. Development and debugging of programs in C/C++ **Independent study of sections of the discipline.** Coding of numerical and symbolic information in computers and basic operations. Concept, properties and methods of description. Development and debugging of programs in C/C++ Description of types. Operators. Operations. Computational operations in C/C++	3	1-12	– review and revision of lecture material; – independent study of sections of the discipline	34	[1]-[3], [8]-[10]
2.	**Independent study of sections of the discipline.** Input/output organization Branching and loops Arrays. Pointers. Strings. Subroutines	4	1-12	– review and revision of lecture material; – independent study of sections of the discipline	72	[1]-[3], [8]-[10]
3.	Working with files. Text and binary files. User-defined data types. Dynamic memory management. Memory management. Scope and lifetime. Preprocessor. Development of multi-module programs and libraries. Build systems. Standard libraries.	5	1-12	– elaboration and repetition of lecture material; – independent study of sections of the discipline	20	[1]-[3], [8]-[10]
4.	Practice 1. **Representation of integer information in a computer.** Problem solving	5	1–12	– review and revision of lecture material; – doing homework	10	[1]-[3], [8]-[10]
5.	Lab 1. **Basics of Algorithmization. Construction of Flowcharts.**	5	1-12	– elaboration and repetition of lecture material; – preparation for laboratory work, preparation of reports on the completion of laboratory work	15	[1]-[3], [8]-[10]

No p/n	Topic of the lesson	Trimester	Completion time (weeks)	Type of independent work	Time Expenditure (Hours)	Educational and methodological support
6.	Practice 2: Loops, **Arrays, Pointers.**	5	1–12	– review and revision of lecture material; – doing homework – passing the test	22	[1]-[3], [8]-[10]
7.	Lab 2. **Branching and loops.**	5	1-12	– elaboration and repetition of lecture material; – preparation for laboratory work, preparation of reports on the completion of laboratory work	15	[1]-[3], [8]-[10]
8.	Lab 3. **Arrays. Pointers. Strings.**	5	1-12	– review and revision of lecture material; – preparation for laboratory work, preparation of reports on the completion of laboratory work	15	[1]-[3], [8]-[10]
	Preparing for the exam				**9**	[1]-[3], [8]-[10]
	Total workload of independent work on the subject				**212**	**-**

4.4. Classes implemented in the form of practical training

No p/n	Topic of the lesson	Type of academic work	Format of the lesson	Location of the lesson	Volume in hours
1.	Practice 1_1 **Representation of integer information in a computer.** Problem solving	Practical lesson	Seminar	SFedU	2
2.	Practice 1_2 **Representation of string information and numeric values in a computer** . Problem solving	Practical lesson	Seminar	SFedU	2
3.	Practice **1_4 Flowcharts. Cycles and Subroutines.** Problem Solving	Practical lesson	Seminar	SFedU	2
4.	Practice **1_3 Flowcharts. Linear Algorithms and Branching.** Problem Solving	Practical lesson	Seminar	SFedU	2
5.	Lab 0: **Integrated tools** for developing and debugging C/C++ programs	Laboratory work	Reproductive laboratory work	SFedU	2

6.	Laboratory work 1. **Presentation of information in a computer and the basics of algorithmization.** Final lesson on topic 1.	Laboratory work	Reproductive laboratory work	SFedU	2
7.	Practice 2_1 **Linear Algorithms**. Data Types. Operators. Operations. Expressions	Practical lesson	Seminar	SFedU	2
8.	Practice 2_2 **Linear Algorithms.** Data Types. Formatted Input/Output	Practical lesson	Seminar	SFedU	2
9.	Lab 2: **Linear Algorithms.** Final Lesson on Topic 2	Laboratory work	Reproductive laboratory work	SFedU	2
10.	Practice 3_1 **Branching and Looping.** Conditional Operator and Switch	Practical lesson	Seminar	SFedU	2
11.	Practice **3_2 Branching and Looping.** Loops with a Counter and a Condition	Practical lesson	Seminar	SFedU	2
12.	Lab 3: **Branching and Looping.** Final Lesson on Topic 3	Laboratory work	Reproductive laboratory work	SFedU	2
13.	Practice 4_1 **Arrays. Pointers. Strings.** Arrays and Pointers	Practical lesson	Seminar	SFedU	2
14.	Practice 4_2 **Arrays. Pointers. Strings.** Strings and Pointers	Practical lesson	Seminar	SFedU	2
15.	Lab 4: **Arrays, Pointers, and Strings.** Final Lesson on Topic 4	Laboratory work	Reproductive laboratory work	SFedU	2
16.	Practice 5_1 **Subroutines.** Data transfer and return.	Practical lesson	Seminar	SFedU	2
17.	Practice 5_2 **Subroutines.** Recursive subroutines and library functions	Practical lesson	Seminar	SFedU	2
18.	Laboratory work 5 **Subroutines** Final lesson on topic 5	Laboratory work	Reproductive laboratory work	SFedU	2
19.	Practice 6_1 **Files.** Text files	Practical lesson	Seminar	SFedU	2
20.	Practice 6_2 **Files.** Binary files	Practical lesson	Seminar	SFedU	2
21.	Lab 6 **Files**. Final lesson on topic 6	Laboratory work	Reproductive laboratory work	SFedU	2
22.	Practice 7_1 **Dynamic memory and user-defined data types.** Dynamic memory. The struct type.	Practical lesson	Seminar	SFedU	2
23.	Pratika7_2 **Dynamic memory and user-defined data types.** Union type. Typedef operator.	Practical lesson	Seminar	SFedU	2
24.	Lab 7: **Dynamic Memory and User-Defined Data Types.** Final Lesson on Topic 7	Laboratory work	Reproductive laboratory work	SFedU	2
25.	Practice 8_1 **Implementation of a linked list.**	Practical lesson	Seminar	SFedU	2
26.	Practice 8_2: **Linked List Implementation. Completion**	Practical lesson	Seminar	SFedU	2

27.	Lab 8. **Solving AI problems based on multithreaded programming and heterogeneous computing.**	Laboratory work	Reproductive laboratory work	SFedU	2
TOTAL:					54

V. EDUCATIONAL AND METHODOLOGICAL SUPPORT OF THE DISCIPLINE

5.1. Main literature

1. Computer Science [Text] : basic course : textbook / ed. by S. V. Simonovich. - 2nd ed. - St. Petersburg. : Piter, 2010. - 640 p. : ill.. - (Textbook for universities). - Bibliogr.: p. 631-632 (28 titles). - ISBN 978-5-94723-752-8 – 47 copies.

2. Andrianova, A. A. Algorithmization and programming. Workshop : textbook / A. A. Andrianova, L. N. Ismagilov, T. M. Mukhtarova. - St. Petersburg : Lan, 2022. - 240 p. — ISBN 978-5-8114-3336-0. - Text : electronic // Lan : electronic library system. — URL: https://e.lanbook.com/book/206258 (accessed: 06.05.2025).

3. Ratseev, S. M. Programming in C language / S. M. Ratseev. - 2nd ed., ster. - St. Petersburg : Lan, 2023. - 332 p. - ISBN 978-5-507-47236-9. - Text : electronic // Lan : electronic library system. - URL: https://e.lanbook.com/book/351863 (accessed: 06.05.2025).

4. Tsarev R. Yu. Programming in C language / R.Yu. Tsarev - Krasnoyarsk: Siberian Federal University, 2014. - 108 p. [Electronic resource]. - URL: http://biblioclub.ru/index.php?page=book&id=364601

5. Williams E. Parallel programming in C++ in action. Practice of developing multithreaded programs. - M.: DMK-Press, 2016. - 672 p. — ISBN 978-5-94074-537-2, 978-5-97060-194-5

6. Posh M. Programming of embedded systems in C++17. - M.: DMK Press, 2020. - 394 p. — ISBN 978-5-97060-785-5

5.2. Further reading

7. Guide to performing a cycle of laboratory works in the discipline "Programming" No. 5044, part 1, Taganrog, Publishing House of Southern Federal University, 2013. - URL: http://ntb.tti.sfedu.ru/UML/UML_5044_1.pdf

8. Kernighan B. W. The C Programming Language / B.W. Kernighan; D.M. Ritchie - Moscow: Internet University of Information Technologies, 2006. - 272 p. [Electronic resource]. - URL: http://biblioclub.ru/index.php?page=book&id=234039

9. Guide to performing a cycle of laboratory works in the discipline "Programming" No. 5044, part 2, Taganrog, Publishing House of Southern Federal University, 2013. - URL: http://ntb.tti.sfedu.ru/UML/UML_5044_2.pdf

10. Programming. Collection of problems : textbook for universities / O. G. Arkhipov, V. S. Batasova, P. V. Grechkina [et al.] ; edited by M. M. Maran. - 3rd ed., ster. - St. Petersburg : Lan, 2025. - 140 p. - ISBN 978-5-507-50516-6. - Text : electronic // Lan : electronic library system. - URL: https://e.lanbook.com/book/443291 (accessed: 06.05.2025).

11. Antonov A.S. Parallel programming using OpenMP. - M.: Moscow State University Publishing House, 2009. - 128 p.

12. Kussvurum D. Modern Parallel Programming with C++ and Assembly Language. - Apress, 2022. - 642 p. - ISBN 978-1-4842-7917-5

13. Djandzhua M. TinyML Cookbook. - Packt, 2023. - 280 p. - ISBN 978-1-80461-512-0`

5.3. Periodicals

– "Programmnaya Inzheneriya" [Software Engineering]. Theoretical and applied scientific-technical journal. ISSN 2220-3397;

– "Informatsionnye Tekhnologii" [Information Technologies]. Scientific-technical and scientific-production journal. ISSN 1684-6400

–

– Itsubo T. et al. An FPGA-based optimizer design for distributed deep learning with multiple GPUs // IEICE Transactions on Information and Systems. — 2021. — Vol. E104D, № 12. — P. 2057–2067. — DOI: 10.1587/transinf.2021EDL8032

Optimizing Neural Networks for Efficient FPGA Implementation: A Survey. — 2021. — Vol. 28. — P. 4537–4547

5.4. List of Internet resources

– Codeforces platform for programming competitions and training. http://www.codeforces.com
– Yandex.Contest platform for programming competitions and training. http://contest.yandex.ru
– "Distance Learning in Informatics" platform. https://informatics.mccme.ru
– Problem archive with Timus Online Judge testing system. http://acm.timus.ru
– Problem archive with "Programmer's School" testing system. https://acmp.ru/index.asp?main=tasks
– INTUIT. Data Structures and Computer Data Processing Algorithms. http://www.intuit.ru/studies/courses/648/504/info
– INTUIT. Fundamentals of Software Development Using the C Language. http://www.intuit.ru/studies/courses/11876/1156/info
– INTUIT. Fundamentals of Programming in C Language. http://www.intuit.ru/studies/courses/43/43/info
– Introduction to C and C++ Programming Languages. http://www.intuit.ru/studies/courses/1039/231/info
– http://ru.stackoverflow.com/
– https://msdn.microsoft.com/en-us/default.aspx
– http://cpp-reference.ru/
– INTUIT. Algorithms and Computation Theory. https://intuit.ru/studies/courses/555/411/info
– INTUIT. Introduction to Schemes, Automata and Algorithms. https://intuit.ru/studies/courses/1030/205/info

5.5 Scientific publications at A* level conferences, articles published in level 1 journals of the White List of Scientific Journals"

1. Sokolov A.P., Makarenkov V.M., Pershin A.Yu., Lanshevsky I.A. Development of software for code generation based on templates when creating engineering analysis systems // Software Engineering. - 2019. No. 9-10. P. 400-416
2. Kostyukhin K.A., Levchenkova G.L., Samborsky S.V. On some innovations of the C23 standard of the C programming language // Software Engineering. - 2025. No. 2. P. 59-71
3. Using the MPI library for parallel implementation of the exhaustive search algorithm // Software products and systems – 2023, No. 36 (4). P. 607-614

VI. LOGISTICAL SUPPORT OF DISCIPLINE

The following premises, equipment and software are used in the implementation of the discipline:

Item No.	Type of classroom lesson	Recommended audience	
		Type	Equipment
1	Lecture classes	lecture-type audience, seminar-type audience, ongoing monitoring, individual and group consultations, midterm assessment	– Projector - 1 pc., Screen - 1 pc. Teacher's computer - 1 pc.; – Microsoft Windows, Microsoft Office PowerPoint
2	Practical classes	computer class	– interactive whiteboard – 1 pc., teacher's computer – 1 pc.; – computers – 25 pcs. with Internet access; – Microsoft Windows, Microsoft Visual Studio, Code Blocks

Item No.	Type of classroom lesson	Recommended audience	
		Type	Equipment
3	Laboratory classes	computer class	– interactive whiteboard – 1 pc., teacher's computer – 1 pc.; – computers – 25 pcs. with Internet access; Microsoft Windows, Microsoft Visual Studio, Code Blocks
4	Independent work	independent study room	
5	Exam	computer class	– interactive whiteboard – 1 pc., teacher's computer – 1 pc.; – computers – 25 pcs. with Internet access; Microsoft Windows, Microsoft Visual Studio, Code Blocks

VII. METHODOLOGICAL GUIDELINES FOR STUDENTS ON MASTERING THE DISCIPLINE

The course "Algorithmization and Programming" is taught in the first semester.

The curriculum includes classroom lectures, practical classes, laboratory classes, and independent work, as well as an exam. The instructor monitors attendance at all classes.

Lectures are delivered with slide presentations.

Practical classes are held in a classroom equipped with an interactive whiteboard. The instructor monitors the solution of each problem. Students are allowed to complete the solutions at home and then submit their solutions in the next practical class.

Independent work includes preparation for laboratory classes and homework assignments.

The goal of practical classes is to develop practical skills—professional or academic—required for subsequent academic activities in general and specialized disciplines. In accordance with the primary didactic goal, the content of practical classes is the solution of various types of problems. During practical classes, students acquire basic professional skills and abilities, which are subsequently reinforced and refined through the study of professional disciplines.

Along with the development of skills and abilities, practical classes generalize, systematize, deepen, and concretize theoretical knowledge, develop the ability and willingness to apply theoretical knowledge in practice, and develop intellectual competencies.

Practical and laboratory classes require prior theoretical preparation on the relevant topic: review of lecture material, study of textbooks, and study of supplementary literature. Students are not required to read all the literature listed in the list of primary and supplementary literature.

Laboratory classes require review of the material from a master class held on this topic during the practical class.

During laboratory classes, students demonstrate their acquired skills and abilities by independently solving problems.

Monitoring of material acquisition is carried out during laboratory classes by reviewing completed problems, homework assignments, and independent and quizzes. Students who, for a valid reason, failed to achieve the required score on the current assessment may, with the instructor's approval, eliminate their outstanding scores before the midterm assessment.

Bonus points are awarded for participation in Olympiads, competitions, and activities related to the curriculum, as well as active participation in lectures and labs.

The final score is converted into a grade as follows:

85-100 = "excellent,"

71-84 = "good,"

60-70 = "satisfactory,"

less than 60 = "unsatisfactory."

VIII. VIII. ASSESSMENT TOOLS FUND

8.1. Passport of the appraisal fund

Item No.	Code of the field of study, specialty	Competency achievement indicator code	Name of the assessment tool
1	09.03.01 09.03.02 09.03.03 09.03.04	OPK-3.1	– Lab work No. 8 (implementation, report preparation, report defense) – exam questions and tickets
2		OPK-4.1	– laboratory works No. 2–7 – exam questions and tickets
3		OPK-4.2	– laboratory works No. 2–7 – exam questions and tickets
4		OPK-4.3	– laboratory works No. 2–7 – exam questions and tickets
5	09.03.04	OPK-6.2	– laboratory work No. 1 – exam questions and tickets
6	09.03.01 09.03.02 09.03.03 09.03.04	PC-10.1 PL-3.1	– laboratory work No. 8 – exam questions and tickets
7		PC-10.2 PL - 3.2	– laboratory work No. 8 – exam questions and tickets
8		PC-10.3 PL-3.3	– laboratory work No. 8 – case assignment – exam questions and tickets

8.2. Laboratory work No. 1

Assessment tool type (assessment subject): Preparation and submission of a laboratory report

List of assessed topics (modules):

Representation of information in a computer and the basics of algorithmization

Mode of study: full-time

Guidelines for conducting laboratory work

Build algorithms for solving the following problems in the Draw.io graphics editor:

a. Given real numbers x and y, find the maximum of them.

b. Given arbitrary numbers a, b, c. If a triangle with sides a, b, c exists, determine whether it is equilateral, isosceles, or scalene. If the triangle does not exist, display the appropriate message.

c. Determine the number of three-digit natural numbers whose sum of digits is equal to a given natural number n. Do not use division operations.

d. Given an array of real numbers, find the value and index of the element in it that is closest to a given integer.

e. Given the coordinates of the vertices of two triangles, find which one has the larger area using the triangle area calculation routine.

Evaluation criteria:

The criterion for assessing admission to laboratory work is knowledge of the theoretical material from previous lectures, necessary for completing assignments in class.

The criterion for completing laboratory work is the flowcharts prepared by students, which they then present, reporting on the results of the completed assignments and answering questions about their work. Points for completed and defended work are awarded based on the objectives achieved and the answers provided.

- 10-11 points are awarded to the student if the work is completed within the established deadlines, the flowchart is designed in accordance with the requirements, the flowchart corresponds to the version of the individual assignment, the student is able to explain the operation of any algorithm;
- 7-9 points are awarded to the student if the work is completed within the established deadlines, the flowchart is designed with minor deviations from the requirements, the flowchart basically corresponds to the version of the individual assignment, the student can explain the operation of any section of the algorithm, allowing for inaccuracies and minor errors;
- 0 points if the assignment is completed outside the established deadlines, the report is prepared with significant deviations from the requirements, the flowchart basically corresponds to the individual assignment version, the student cannot explain any part of the algorithm.

8.3. Laboratory work No. 2

Assessment tool type (assessment subject): performance of laboratory work

List of assessed topics (modules):

Linear algorithms.

Mode of study: full-time

Guidelines for conducting laboratory work
The lab involves independently writing a program code that solves a given problem. The solution is verified using an automated solution verification system. Each student is assigned 2-4 problems and is given 90 minutes to solve them. The students then upload their completed solutions to the verification system.

Practical assignment assessment criteria:
The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.4. Laboratory work No. 3

Assessment tool type (assessment subject): performance of laboratory work

List of assessed topics (modules):

Branching and loops.

Mode of study: full-time

Guidelines for conducting laboratory work
The lab involves independently writing a program code that solves a given problem. The solution is verified using an automated solution verification system. Each student is assigned 2-4 problems and is given 90 minutes to solve them. The students then upload their completed solutions to the verification system.

Practical assignment assessment criteria:
The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.5. Laboratory work No. 4

Assessment tool type (assessment subject): performance of laboratory work

List of assessed topics (modules):

Arrays. Pointers. Strings.

Mode of study: full-time

Guidelines for conducting laboratory work
The lab involves independently writing a program code that solves a given problem. The solution is verified using an automated solution verification system. Each student is assigned 2-4 problems and is given 90 minutes to solve them. The students then upload their completed solutions to the verification system.

Practical assignment assessment criteria:
The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.6. Laboratory work No. 5

Assessment tool type (assessment subject): performance of laboratory work

List of assessed topics (modules):

Subroutines.

Mode of study: full-time

Guidelines for conducting laboratory work
The lab involves independently writing a program code that solves a given problem. The solution is verified using an automated solution verification system. Each student is assigned 2-4 problems and is given 90 minutes to solve them. The students then upload their completed solutions to the verification system.

Practical assignment assessment criteria:
The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.7. Laboratory work No. 6

Assessment tool type (assessment subject): performance of laboratory work

List of assessed topics (modules):

Files.

Mode of study: full-time

Guidelines for conducting laboratory work
The lab involves independently writing a program code that solves a given problem. The solution is verified using an automated solution verification system. Each student is assigned 2-4 problems and is given 90 minutes to solve them. The students then upload their completed solutions to the verification system.

Practical assignment assessment criteria:

The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.8. Laboratory work No. 7

Assessment tool type (assessment subject): performance of laboratory work

List of assessed topics (modules):

Dynamic memory and user-defined data types.

Mode of study: full-time

Guidelines for conducting laboratory work

The lab involves independently writing a program code that solves a given problem. The solution is verified using an automated solution verification system. Each student is assigned 2-4 problems and is given 90 minutes to solve them. The students then upload their completed solutions to the verification system.

Practical assignment assessment criteria:

The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.9. Laboratory work No. 8

Assessment tool type (assessment subject): performance of laboratory work

List of assessed topics (modules):

Solving AI problems using multithreaded programming and heterogeneous computing.

Mode of study: full-time

Guidelines for conducting laboratory work

Target:
Master the principles of parallel computing and the use of OpenCL technologies to accelerate machine learning algorithms.
Exercise:
Implement one of the following algorithms in C++ using OpenCL:

- linear regression (training a model on a data set and calculating coefficients),
- k-means clustering (initialization of centers, iterative distribution of points and updating of centers).

Result:
A program that runs calculations on a graphics processing unit (GPU) or other OpenCL computing device; speedup analysis compared to the CPU version.

Practical assignment assessment criteria:

The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.10. Computer Science and Programming Case Study

Assessment tool type (assessment subject): case solution

List of assessed topics (modules):

Standard algorithms and data structures.

Mode of study: full-time

The tasks were compiled with the participation of representatives of the IT company.

Methodological recommendations for conducting
The lab involves independently writing a program code that solves a given problem. The solution is verified using an automated solution verification system. Each student is assigned several problems and is given 90 minutes to solve them. The students then upload their completed solutions to the verification system.

Criteria for evaluating the case solution:
The student's solution is checked on a test set unavailable to the participant. For each assignment, the student can receive up to 100 raw points. These raw points are then converted into rating points proportionally and rounded to the nearest whole number according to mathematical rules. Points earned for lab work may be cancelled if plagiarism or outside assistance is detected in solving the problem. After recalculation, a student can receive a maximum of 4 points for completing all laboratory tasks.

8.11. Test

Type of assessment tool (subject of control): test paper

List of controlled topics (modules):

The whole course.

Mode of study: correspondence

Methodological recommendations
The test is administered in the LMS and covers topics covered throughout the semester. The maximum score is 15. Completion time is 60 minutes. No supplementary materials are permitted during the test. The test includes 15 questions, each of which has one correct answer.

Test evaluation criteria:
The test consists of 15 questions. Each question, if answered correctly, is worth 1 point. The maximum score for the entire test is 15. The test is passed if a student scores 9 points.
Example of test questions:

1. Which of the following is the correct definition of a function that returns a pointer to an array of integers?			MC
#	**Answers**	**Review**	**Grade**
A.	int* func();		*0*
B.	int (*func())[10];		*100*
C.	int *func()[10]		*0*
D.	int[10] *func();		*0*
E.	int func()[10]		*0*

2. Which of the following is the correct way to pass a two-dimensional array to a function?			MC
#	**Answers**	**Review**	**Grade**
A.	void func(int arr[][], int rows, int cols);		*0*
B.	void func(int arr[10][10]);		*0*
C.	void func(int **arr, int rows, int cols);		*0*
D.	void func(int *arr[], int rows, int cols);		*0*
E.	void func(int (*arr)[10], int rows);		*100*

3. The result of the program will be

```cpp
#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 1, y = 2;
    swap(&x, &y);
    cout << x << " " << y << endl;
    return 0;
}
```

MC

#	**Answers**	**Review**	**Grade**
A.	1 2		*0*
B.	2 1		*100*
C.	0 0		*0*
D.	12		*0*
E.	21		*0*

4. The result of the program will be

```cpp
#include <iostream>
using namespace std;

void modify(int *arr, int size) {
    for(int i = 0; i < size; ++i) {
        arr[i] = arr[i] * 2;
    }
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    modify(arr, 4);
    for(int i = 0; i < 5; ++i) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

MC

#	Answers	Review	Grade
A.	1 2 3 4 5		0
B.	2 4 6 8 10		0
C.	1 4 9 16 25		0
D.	2 4 6 8 5		100
E.	2 4 6 8		0

5. What is the purpose of the main() function in C++?

MC

#	Answers	Review	Grade
A.	Perform any task in the program		0
B.	Serve as an entry point for the program		100
C.	Define all global variables		0
D.	Define all classes		0
E.	Serve as an exit point from the program		0

6. $_{10}$ to binary Do not include leading zeros.			SA
		Default score:	1
		Case sensitivity:	No
		Penalty for each incorrect attempt:	33.3
		ID number:	
	Answers	**Review**	**Grade**
	1101		*100*

7. Find the value of a variable
`#include <iostream>`
`using namespace std;`
`int main(){`
`  int arr[] = {1,2,3,4,5};`
`  int sum = 0;`
`  for(int i = 1; i < 5; i+= 2){`
`  sum += arr[i];`
`  }`
`  cout << sum;`
`  return 0;`
`}`
			MC

#	**Answers**	**Review**	**Grade**
A.	5		*0*
B.	6		*100*
C.	7		*0*
D.	8		*0*

8. The result of the program will be
`#include <iostream>`
`using namespace std;`
`int main(){`
`  int arr[] = {5, 10, 15, 20};`
`  cout << arr[4];`
`  return 0;`
`}`
			MC

#	**Answers**	**Review**	**Grade**
A.	20		*0*
B.	Random value or runtime error		*100*
C.	The program will complete successfully without output.		*0*
D.	5		*0*

8.12. Exam questions and tickets

Assessment Period: 1st Semester/5th Trimester

Assessment Type (Assessment Subject): Exam

List of Assessed Topics (Modules):

1. Information Representation in Computers: Unsigned and Signed Numbers, Floating-Point Numbers, Symbolic Information. Limitations Associated with Finite-Bit Representation of Numeric Data. Examples

2. The Concept of an Algorithm and Its Basic Properties. Forms of Algorithm Representation. Basic Algorithmic Constructs and Methods of Representing Them in Algorithm Descriptions. Examples.

3. Principles of Structured Programming. The Concept of a Block. A Block (Modular) Approach to Constructing Problem-Solving Algorithms in C/C++. Named and Unnamed Blocks. Examples.

4. Program Structure in C/C++. Keywords, Separators, Identifiers, Comments: Purpose and Examples of Use. The Concept of Declaration and Definition of a Program Object in C/C++. Examples. 5. Variables: Declarations and Initialization. Variable Attributes. Constants: Methods of Setting Them in a Program. The Concept of a Constant Type. Named and Literal Constants. Examples.

6. Numeric Data Types in C/C++. Type Modifiers. Algebraic Expressions and Rules for Writing Them in C/C++. Examples.

7. Symbolic Data Type. ASCII Table. Enumerations. Boolean Data Type. Void Type. Examples of Operations on Corresponding Data Types.

8. Type Compatibility and Conversion (Cast). Explicit and Implicit Conversion. Hierarchy of Basic Data Types during Type Conversion. Examples.

9. The Concept of Operation, Expression, and Operator. Types of Operations Based on the Number of Operands. Compact Notation. Examples.

10. Unary Operations. Prefix and Postfix Notations for Increment and Decrement Operations. Potential Problems. Examples.

11. Relational (comparison) operators: rules for writing and evaluating. Conditional logical operators: rules for writing and evaluating; application to forming compound logical expressions. Examples.

12. The assignment operator in C/C++: rules for writing and evaluating. The concept of "lazy evaluation." Examples.

13. The order of operations in C/C++. The precedence table and its use. Examples.

14. Formatted input and output functions in console mode in C or C++ (optional). Input/output format modifiers. Examples.

15. Branching operators (conditional operator, choice operator) in C/C++. Nested branches. Possibility of replacing the conditional operator with a ternary operation. Examples.

16. Loops with pre- and post-conditions in C/C++. Examples.

17. The for loop in C/C++: writing methods. The for loop as a loop with a parameter. Converting a for loop to a loop with a pre- or postcondition. Examples.

18. Jump operators (break, continue, goto) in C/C++: purpose, uses, limitations. Examples.

19. The concept of an array. Organizing data storage and working with one-dimensional arrays in C/C++. Examples.

20. Organizing data storage and working with two-dimensional arrays in C/C++. Examples.

21. The concept and methods of describing a pointer in a C/C++ program. Permissible operations on pointers. Untyped pointer. Examples.

22. The concept of address arithmetic. Advantages and disadvantages of using address arithmetic compared to indexed addressing. Examples.

23. Storing string information in memory in a C/C++ program. The dual nature of strings. Input/output functions for characters and strings. Examples of library functions for working with strings and characters. Examples.

24. Functions in C/C++. Description and calling in a program. Function prototype and its purpose. Formal and actual parameters. Correspondence between parameters. Examples.

25. Functions in C/C++. Methods of passing parameters to a function, variable parameters, value parameters. Methods of returning values from a function. Examples.

26. The concept of an inline function. The main() function. Accessing command-line parameters when running a program. Examples.

27. The concept of recursion. The recursion mechanism. The structure of a recursive function. The possibility of converting between recursive and iterative forms of an algorithm. Examples.

28. Function pointer: declaration, capabilities. Examples.

29. The concept of a stream. Sequential and arbitrary methods of accessing stream data. Data buffering during input and output. Standard Streams. Examples.

30. Files in C/C++. The FILE structure. Functions for opening and closing files. File opening modes. Differences in working with text and binary files. Examples.

31. Formatted text file input/output in C or C++ (optional). Determining end-of-file. Examples.

32. Working with binary files with C/C++. Functions for reading/writing binary data blocks to a file. Examples.

33. The stream pointer in C/C++. Library functions for moving around a file. Redirecting input/output streams. Examples.

34. Combined/composite data types in C/C++. Structure (struct): ways to declare a type and declare variables of this type. Pointer to a structure. Accessing structure fields. Operations on structures. Examples.

35. Storing structures in memory in C/C++. Field alignment. Determining the size of a structure. Array of structures: declaration, accessing elements. Pointer to an array of structures. Examples.

36. Unions in C/C++: description, purpose, and difference from structures (structs). Bit fields: purpose, limitations. Defining a new data type name (typedef). Examples.

37. Types of memory used in a C/C++ program: static, stack, dynamic (heap). Functions/methods for working with dynamic memory in a C or C++ program (optional). The problem of memory leaks. Examples.

38. The concept of scope and lifetime of program objects in C/C++. Default lifetime and scope for variables and functions. Purpose of the register, static, and extern modifiers. Examples.

39. The preprocessor in C/C++: purpose and operating principle. Macro definitions and macro substitutions. Differences between macros and functions. Examples.

40. The feasibility of dividing C/C++ program text into modules. Header files: purpose and recommended contents. The problem of multiple variable declarations in header files and how to solve it. Examples.

Guidelines for conducting the exam

The exam is conducted in a mixed format, including theoretical and practical parts.

When conducting the exam, classrooms with the equipment specified in paragraph 6 of the RPD are used.

The exam process is carried out in several stages:

1. Written answer to one theoretical question on topics 1-7. Maximum score: 8. Completion time: 30 minutes. No additional materials may be used when preparing your answer.
2. Complete a quiz in the LMS covering the topics covered during the semester. Maximum score: 12. Completion time: 60 minutes. No supplementary materials are permitted during the quiz.
3. Completion of 3 practical problems using a system with automatic solution checking. Maximum score: 20. Completion time: 150 minutes. Use of lecture presentations and the C/C++ language reference website is permitted when completing the problems.

Criteria for assessing the theoretical question:

- A student is awarded 7-8 points if they have covered the theoretical question, demonstrated solid knowledge, and provided an example of its use. Minor inaccuracies in the answer are allowed.
- 5-6 points are awarded to students who have fully addressed the theoretical issue and provided a practical example. There may be errors in the answer to the question or example, but a solid understanding of the issue is present.
- 3-4 points are awarded to a student if the student has covered the theoretical question, but made mistakes in answering the question and did not give an example.
- 0-2 points are awarded to a student if the student made serious errors in answering the question, did not provide an example of use, or did not understand the essence of the question.

Test evaluation criteria:

The test includes 24 questions, three on each topic. Each test question, if answered correctly, is worth 0.5 points.

Practical assignment assessment criteria:

The practical assignment involves solving three problems using an automated assessment system. The student's solution is evaluated on a test set unavailable to the participant. For each problem, the student can receive up to 100 initial points. These initial points are then converted into exam points proportionally and rounded to the nearest whole number according to mathematical rules. Exam points may be cancelled if plagiarism or third-party assistance is detected in solving the problem.
For solving the first problem, the student can receive a maximum of 4 points.
For solving the second problem, the student can receive a maximum of 6 points.
For solving the third problem, the student can receive a maximum of 10 points.